F6-IC few-candidate closure, a parallel split of one target's tree, and the re-reduction cost (2026-11-05)

Three follow-ups to `research/f4_f5_followups_20261029`:

1. prune more of the F6-IC search tree;
2. stop re-reducing the rows a node inherits from its parent;
3. solve one target's decision tree on several threads.

Report: `REPORT.pdf` (source `REPORT.typ`). Graphs:

- `reports/run-20261105T1820Z/fig_few_candidates.svg`;
- `reports/parallel-20261105T1911Z/fig_parallel.svg`.

Host: shared 4-vCPU Xeon @ 2.10 GHz, unpinned (L0). One frozen binary per run, its sha256 in each run's `binary.sha256`. Arms differ by environment only and alternate their order each round.

1. Few-candidate closure in the F6-IC gate: on by default

Change (`src/cryptanalysis/koblitz_index_calculus.rs`, `F6GeometricGate::decide`).

Before, for a three-summand chain using the pair index, the gate decided a node exactly only once one summand's coordinate bits were all assigned. It then tried that summand's point lifts against the pair-sum index (the one-fixed closure).

Now, as soon as a summand's assigned bits leave at most `KIC_F6_FEW_CANDIDATES` (default 64) base codes, the gate runs that same exact closure for each candidate x-coordinate and decides the whole subtree:

- **Witness** if one closes the group identity;
- **Refute** if none does.

`KIC_F6_FEW_CANDIDATES=0` is the same-binary control.

Run `run-20261105T1820Z`: nine Koblitz cells, 5 rounds × 3 repetitions, arms `on` and `off` (`scripts/run_arms.sh`; `summarize.py`, `plot.py`).

Verdicts and indices	identical in both arms and every round: 80 target × engine pairs, 10 decomposing targets among them
Gate pair lookups and group additions	identical in both arms, on all 40 F6-IC targets
F6-IC, CPU off ÷ on	**2.67–10.29**× (median of 5); the worst pairing of rounds is still ≥ 1.66×
F6-IC, word XORs off ÷ on	1.84–10.13×
F6-IC nodes	204–1,049 reductions per cell become 5–16
Control: inherited F4 (no gate), same binary	word XORs identical; CPU 0.87–1.05×

What this means. The pair lookups and group additions are the same with and without the closure, so on these cells the algebra was not ruling out any candidate x-coordinate. With the pair index, F6-IC on these cells works as the pair-index meet-in-the-middle scan: the closure removes the algebra around that scan.

- It is a real reduction in work for the `f6-ic` method: fewer word XORs, the same group operations.
- It does not beat the meet-in-the-middle oracle the repository already has (ecbench's `mitm`), because in the limit it is that oracle.

Accounting.

- ecbench sessions now record `f6_ic_tree = 2` in their accounting versions.
- A failed replay of an older `f6-ic` record names the tree change (`record::f6_tree_note`) instead of reporting bare count differences.

Tests. `f6_ic_few_candidate_closure_matches_exact_enumeration` (new) checks the closure against exhaustive enumeration. The existing F6-IC, Koblitz index-calculus, solver and ecbench tests pass.

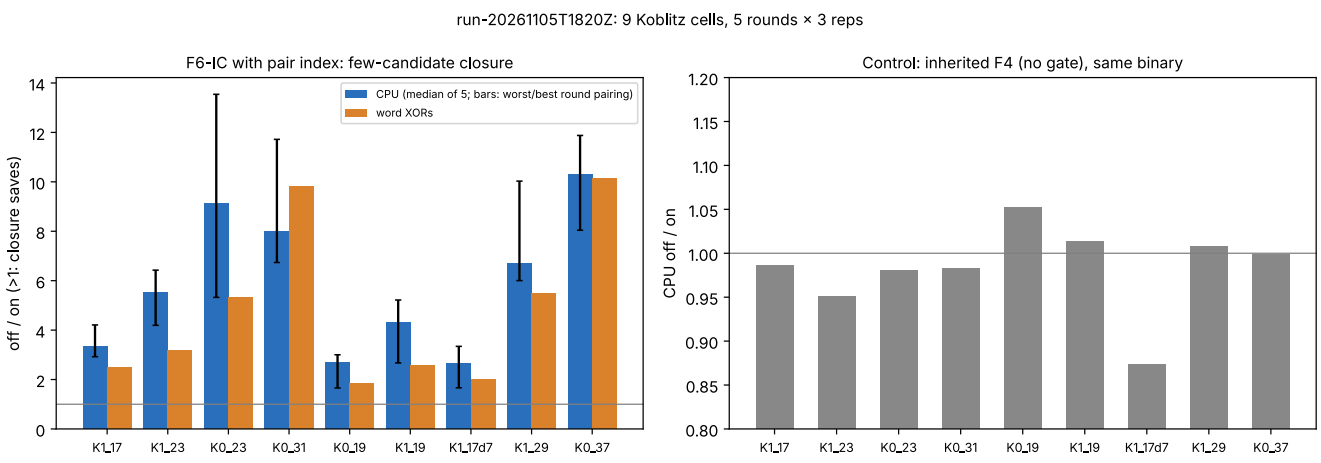


Figure 1: Item 1, run-20261105T1820Z: CPU and word-XOR ratio off ÷ on per cell (5 rounds), and the inherited-F4 control.

2. Re-reducing displaced rows: no change, measured why

On the plain inherited-F4 engine, the tree below the root is 67–83 % of word XORs on the larger cells. A temporary counter (not committed) measured a node's displaced rows and the pivots each crosses:

cell	displaced rows / node (median, mean, max)	words / row	pivots crossed / displaced row (median, approx.)
K0_23	36, 53, 275	29	7.9
K0_37	61, 99, 702	62	24.5
K1_29	48, 76, 434	40	13.8

- **Batching the rows (Four Russians).** A batch over a basis of a few hundred pivots spends about 32 row operations per pivot on its tables. That is several times the roughly 1,500 row XORs per node it would replace, so the batch costs more than it saves here.
- **Replaying root shapes.** Recording each root shape's dependent rows and dropping them from later roots of the same shape is already a recorded loss (`research/pdp_shape_replay_20261015`, 0.75–0.99×).
- **Reduced form after each level.** Restoring it was rejected earlier (`RESEARCH_INHERITED_F4.md` §3.5).

No design was found that does less work, so nothing is changed.

3. Parallel split of one target's tree: opt-in

Change (`src/cryptanalysis/koblitz_groebner.rs`). `KIC_SOLVE_PARALLEL_DEPTH=k` (default 0) runs the first `k` split levels of a solve without a node oracle as `2^k` independent subtrees in parallel:

- **Bases.** Each child builds its own bases, because the inherited bases are single-threaded.
- **Accepting a root.** Each child explores its whole subtree. The parent offers their roots to `accept` in the sequential order, and stops at the first one it accepts.
- **Counted work.** Each worker's per-thread work counters are credited back to the calling thread.
- **Ecbench.** It refuses `KIC_*` overrides, so this stays out of stage-unit measurements.

Test. `parallel_split_returns_the_sequential_roots_in_order` checks depths 1–3 against the sequential solve:

- the roots are identical;
- they come back in the same order;
- the solve stops at the same accepted root;
- the workers' work is credited to the calling thread.

Run `parallel-20261105T1911Z`: inherited F4, `RAYON_NUM_THREADS=4`, depth 0 against depth 2, nine cells, 5 rounds (`scripts/run_parallel.sh`, `summarize_parallel.py`, `plot_parallel.py`).

Verdicts and indices	identical on all 40 targets in both arms and every round
Wall time per target, sequential ÷ parallel, cells with no decomposition	**1.59–2.88×**
Same, cells where most targets decompose (K0_19: 3/4; K1_17d7: 6/8)	0.94× and 1.11×
Counted work, parallel ÷ sequential	1.12–1.49× on refuting cells; 2.67× and 2.86× on the decomposing ones

The decomposing cells lose because a sequential solve stops at its first accepted root, while each parallel child finishes its whole subtree. The split pays where the cost is a whole refutation, which is most index-calculus targets. It is opt-in because it changes the counted work.

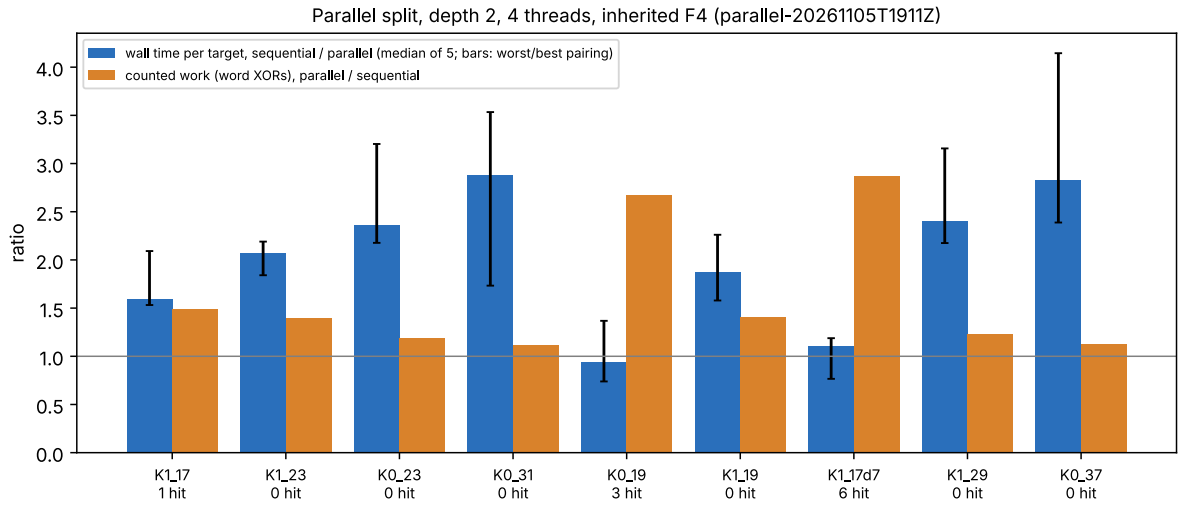


Figure 2: Item 3, parallel-20261105T1911Z: wall time per target, sequential ÷ parallel (depth 2, 4 threads), and the extra counted work.

Not claimed

- No end-to-end index-calculus speed-up and no comparison against rho. Item 1 speeds up the `f6-ic` arm, and in the limit that arm is the existing meet-in-the-middle oracle.
- No `benchmarks/README.md` or leaderboard row: no replay certificate, validator report or rho reference has been produced (validation gate). Copies are in `benchmarks/20261105-f6-few-candidates/`.
- Local development measurements, not an Iso Lab run.